

Java

Ereditarietà

RIUSO DEL CODICE

- Durante lo sviluppo frequentemente i programmatori scrivono codice *molto simile* a codice già esistente
- Questo, spesso, viene fatto manipolando il codice esistente con operazioni di tipo "cut" & "past"
- Si vuole riusare tutto ciò che può essere riusato (componenti, codice, astrazioni)
- Non è utile né opportuno modificare codice già funzionante e corretto il cui sviluppo ha richiesto tempo (anni-uomo) ed è costato (molto) denaro
- Occorre
 - un modo per catturare le similitudini e formalizzarle
 - un linguaggio che consenta di progettare codice in grado di:
 - progettare le differenze
 - sviluppare codice in modo incrementale.

Inheritance

- L'ereditarietà consente di riutilizzare in modo vantaggioso una classe già definita che è simile a quella che vogliamo definire
- Consente di utilizzare il **polimorfismo**!
- La nuova classe è chiamata "**sottoclasse**"
- Attraverso l'**estensione** della classe pre-esistente (chiamata "**superclasse**"), noi possiamo:
 - aggiungere **nuovi dati (attributi)** a quelli presenti nella superclasse
 - aggiungere **nuovi metodi** a quelli presenti nella superclasse
 - ridefinire alcuni metodi della superclasse secondo le nuove esigenze (**Overriding dei metodi**!)

EREDITARIETÀ

- Una relazione tra classi:
 - si dice che la nuova classe B (CLASSE DERIVATA o SOTTOCLASSE) eredita dalla preesistente classe A (CLASSE BASE o SUPERCLASSE)
- La nuova classe che ESTENDE un classe già esistente
 - può aggiungere nuovi dati o metodi
 - può accedere ai dati ereditati purché il livello di protezione lo consenta
 - non può eliminare dati o metodi.
- La classe derivata condivide la struttura e il comportamento della classe base
- Attenzione al termine “superclasse”
 - non vuol dire “classe super”, ovvero classe dotata di maggiori funzionalità ma “classe che sta sopra nella gerarchia”
- Viceversa
 - tipicamente sono gli oggetti delle sottoclassi ad avere maggiori funzionalità (attributi e metodi) rispetto a quelli delle superclassi

Diverse terminologie

SuperClasse

Classe padre

Classe Base

```
public class SubClass extends SuperClass
{
    // ...
}

```

Sotto Classe

Classe Figlio

Classe Derivata

```
public class Point {  
    private int x;  
    private int y;                                // attributi  
  
    public Point() {                               // costruttore 1  
        this(0,0);  
    }  
    public Point(int x, int y) {                   // Costruttore 2  
        setX(x);  
        setY(y);  
    }  
    public void setX(int x) {                       // modificatore per x  
        this.x = x;  
    }  
    public int getX() {                             // accesso a x  
        return x;  
    }  
  
    // supponiamo esistano i metodi setY e getY  
}  
// class Point
```

Esempio

```
class NewPoint extends Point {  
    public String toString(){return "coordinata x=" +x ;}  
  
}  
  
class Prova {  
    public static void main(String args[]) {  
        NewPoint pc = new NewPoint();  
        pc.setX(42);  
        System.out.println(pc.getX() + " " + pc.getY());  
    }  
}
```

Minimizzare la ridondanza

```
public class LabeledPoint extends Point {  
    String name;  
  
    public LabeledPoint(int x, int y, String name) {  
        setX(x);  
        setY(y);  
        setName(name);  
    }  
    public void setName(String name) {  
        this.name = name;  
    }  
    public String getName() {  
        return name;  
    }  
}
```

EREDITARIETÀ E PROTEZIONE

- Problema: il livello di protezione private impedisce a chiunque di accedere al dato, anche a una classe derivata
 - va bene per dati “veramente privati”
 - ma è troppo restrittivo nella maggioranza dei casi
- Per sfruttare appieno l’ereditarietà occorre rilassare un po’ il livello di protezione
 - senza dover tornare per questo a public
 - senza dover scegliere per forza la protezione package di default: il concetto di package non c’entra niente con l’ereditarietà!
- La **QUALIFICA protected**
 - è come package (il default) per chiunque non sia una classe derivata
 - ma consente libero accesso a una classe derivata, indipendentemente dal package in cui essa è definita.

ESEMPIO

■ Dal contatore (solo in avanti) ...

```
public class Counter {  
    private int val;  
    public Counter() { val = 1; }  
    public Counter(int v) { val = v; }  
    public void reset() { val = 0; }  
    public void inc() { val++; }  
    public int getValue() { return val;}  
}
```

ESEMPIO

Ma val era privato!!

ERRORE

**nessuno può accedere a dati e metodi
privati di qualcun altro!**

**Per sfruttare appieno l'ereditarietà
occorre**

**rilassare un po' il livello di protezione
senza dover tornare per questo a public**

ESEMPIO: una soluzione

... al contatore avanti/indietro (con decremento)

```
public class Counter2 extends Counter  
{  
    public void dec() {  
        int x = getValue();  
        reset();  
        for (int i=0; i<x-1;i++)  
            inc();  
        }  
}
```

Esempio corretto

```
public class Counter {  
    protected int val;  
    public Counter() { val = 1; }  
    public Counter(int v) { val = v; }  
    public void reset() { val = 0; }  
    public void inc() { val++; }  
    public int getValue() { return val;}  
}
```

Esempio corretto: il contatore con decremento

```
public class Counter2 extends Counter {  
    public void dec() { val--; }  
}
```

- La qualifica `protected`:
 - rende accessibile un campo a tutte le sottoclassi, presenti e future
 - costituisce perciò un permesso di accesso “indiscriminato”, valido per ogni possibile sottoclasse che possa in futuro essere definita, senza possibilità di distinzione.
 - I membri `protected` sono citati nella documentazione prodotta da Javadoc (a differenza dei membri qualificati privati o con visibilità `package`).

Ereditarietà e Scope

Ereditarietà e Scope

■ Attributi:

- Java prima esamina il metodo corrente , controllando variabili locali e parametri formali...
- ... quindi esamina la classe *corrente*...
- .. poi esamina la *superclass*..
- ...infine continua nella gerarchie delle classi fino a quando non ci sono piu' superclassi da esaminare.

■ Metodi

■ Java esamina la classe *corrente*...

- ... quindi esamina la *superclasse*...
- ...e continua nella gerarchie delle classi fino a quando non ci sono piu' superclassi da esaminare

Specifica dell'ambiente

- Java consente di superare le regole di ambiente per attributi e metodi utilizzando:
 - Keyword *super*:
 - keyword per specificare metodi e attributi della superclasse, e.g., *super.nomemetodo ()*
 - Keyword *this*:
 - keyword for specificare metodi e attributi dell'oggetto *corrente*, e.g., *this.nomemetodo()*;

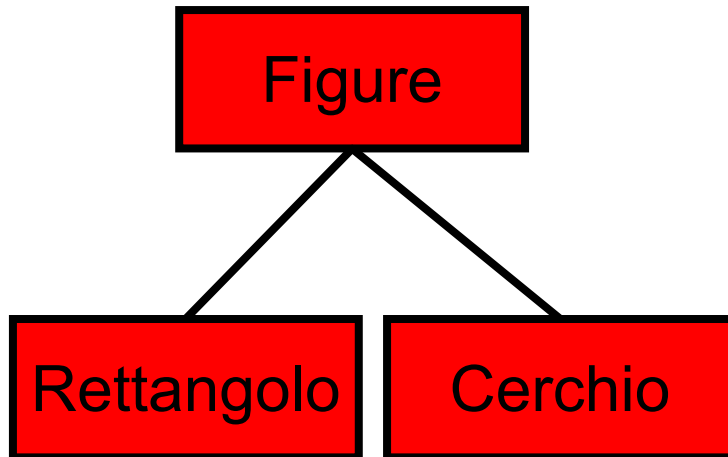
Ereditarietà

```
public class Nonno {  
    public void method1() {...}  
}
```

```
public class Padre extends Nonno {  
    public void method1() {...}  
    public void method2() {...}  
}
```

```
public class Figlio extends Padre {  
    public void method1() {...}  
    public void method3() {  
        method1();  
        method2();  
        super.method1();  
    }  
}
```

A geometry example



**Ogni classe derivata
implementerà il metodo
area nel modo appropriato.**

```
public class Figure {

    public String name;
    public Figure() {}
    public Figure(String s) {
        name = s;
    }

    public String getName () {
        return (this.name);
    } // getName

    public int area () {
        return (0);
    } // area

} // Shape
```

A geometry example

```
public class Rettangolo extends Figure {  
    private int base, altezza;  
  
    public Rettangolo() { this(0, 0); } // costruttore  
  
    public Rettangolo(int base, int altezza) {  
        this(base, altezza, "rettangolo");  
    } // costruttore  
  
    public Rettangolo(int base, int altezza, String name) {  
        this.base = base;  
        this.altezza = altezza;  
        this.name = name;  
    } // costruttore  
  
    public int area() { return (base * altezza); } // area
```

A geometry example

```
public String getName () {  
    if (base == altezza) return "quadrato: " + super.getName();  
    else return (super.getName());  
} // getName  
  
public String toString () {  
    String answer;  
    answer = new String("Il rettangolo chiamato " +  
        getName() + " con altezza " + altezza +  
        " e base " + base);  
    return (answer);  
} // toString  
  
} // Rettangolo
```

A geometry example

```
super(xxx)          // chiama il costruttore della superclasse
super.xxx           // accede agli attributi della superclasse
super.xxx( )        // chiama i metodi della superclasse

this(xxx)           // chiama il costruttore della classe corrente
this.xxx            // accede agli attributi della classe corrente
this.xxx( )         // chiama i metodi della classe corrente
```

Nota: non si puo' super.super<something>

Ereditarietà

- Cosa si eredita?
 - tutti i dati della classe base
 - anche quelli privati, a cui comunque la classe derivata non potrà accedere direttamente
 - tutti i metodi
 - anche quelli che la classe derivata non potrà usare direttamente
 - tranne i costruttori, perché sono specifici di quella particolare classe.

Ereditarietà e costruttori

- Una classe derivata non può prescindere dalla classe base, perché ogni istanza della classe derivata comprende in sé, indirettamente, un oggetto della classe base.
- Quindi, ogni costruttore della classe derivata deve invocare un costruttore della classe base affinché esso costruisca la “parte di oggetto” relativa alla classe base stessa:
 - “ognuno deve costruire ciò che gli compete”

Ereditarietà e costruttori

- Perché bisogna che ogni costruttore della classe derivata invochi un costruttore della classe base?
 - solo il costruttore della classe base può sapere come inizializzare i dati ereditati in modo corretto
 - solo il costruttore della classe base può garantire l'inizializzazione dei dati privati, a cui la classe derivata non potrebbe accedere direttamente
 - è inutile duplicare nella sottoclasse tutto il codice necessario per inizializzare i dati ereditati, che è già stato scritto.

Ereditarietà e costruttori

- Se la prima linea del costruttore non è una chiamata esplicita al costruttore della superclasse, Java, implicitamente, aggiungerà ***super()*** come prima linea.

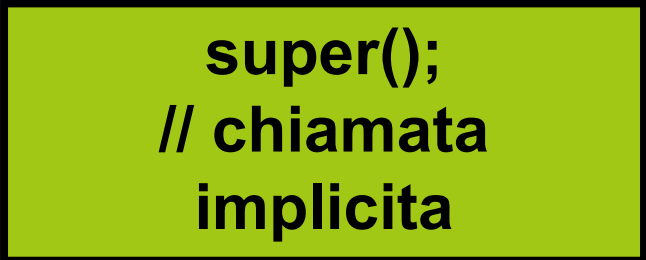
```
public Rettangolo() {  
    .....  
} // of constructor
```

Implica una chiamata a `Figura()` attraverso `super()`;
Si deve essere certi che la superclasse abbia un costruttore di default

- ***Un'eccezione a questa regola:***
 - *non verrà richiamato `super()` solo se c'è un riferimento a `this(params)`*
 - una chiamata esplicita ad un costruttore della superclasse

Ereditarietà e costruttori

```
public class Rettangolo extends Figure {  
    private int base, altezza;  
  
    Rettangolo() { this(0, 0); } // costruttore  
  
    Rettangolo(int base, int altezza)  
        {this(base, altezza, "rettangolo");  
    } // costruttore  
  
    Rettangolo(int base, int altezza, String name) {  
        this.base = base;  
        this.altezza = altezza;  
        this.name = name;  
    } // costruttore  
  
    public int area() { return (base * altezza); } // area
```



ESEMPIO: il contatore con decremento

```
public class Counter2 extends Counter {  
    public void dec() { val--; }  
    public Counter2() { super(); }  
    public Counter2(int v) { super(v); }  
}
```

Costruttore di default generato automaticamente dal sistema in assenza di altri costruttori

L'espressione `super(...)` invoca il costruttore della classe base che corrisponde come numero e tipo di parametri alla lista data.

Ereditarietà e costruttori

- E se non indichiamo alcuna chiamata a `super(...)`?
- Il sistema inserisce automaticamente una chiamata al costruttore di default della classe base aggiungendo la chiamata a `super()`

RICORDARE

il sistema genera automaticamente il costruttore di default solo se noi non definiamo alcun costruttore.

Se c'è anche solo una definizione di costruttore data da noi, il sistema assume che noi sappiamo il fatto nostro, e non genera più il costruttore di default automatico.

Costruttori e protezione

- Di norma, i costruttori sono public
 - in particolare, è sempre pubblico il costruttore di default generato automaticamente da Java
 - almeno un costruttore pubblico deve sempre esistere, a meno che si voglia impedire espressamente di creare oggetti di tale classe agli utenti “non autorizzati”
 - caso tipico: una classe che fornisce solo costruttori protetti è pensata per fungere da classe base per altre classi più specifiche
 - non si vuole che ne vengano create istanze.

Dipendenze d'ordine dei costruttori

- Invocazione di un costruttore della superclasse
- Vengono inizializzati gli attributi mediante i rispettivi inizializzatori
- Viene eseguito il corpo del costruttore
- Se nel corpo del costruttore c'è una chiamata a `this()` si segue la catena di queste invocazioni fino a quando non viene incontrata una chiamata implicita o esplicita di un costruttore della superclasse

Ereditarietà e ridefinizione dei membri: i metodi

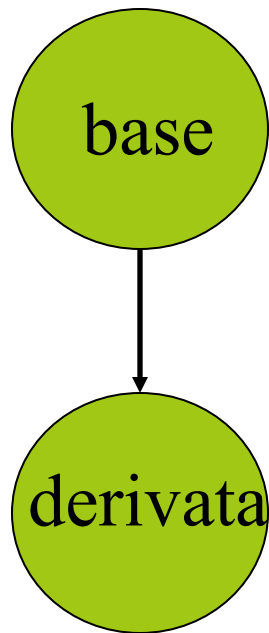
- L'overloading dei metodi consiste nell'avere a disposizione più metodi dotati dello stesso nome, ma con diversa signature per poterli distinguere
- L'overriding di un metodo consiste nel sostituire all'implementazione di un metodo della superclasse un'implementazione nuova, le signature dei metodi devono rimanere le stesse.
- Una sottoclasse può ridefinire i modificatori di accesso di un metodo solo per renderlo più ampio
- Nella ridefinizione dei metodi è possibile variare i modificatori `synchronized`, `native` e `strictfp`.
- Non è possibile ridefinire un metodo `final` mentre un metodo ridefinito può essere `final`
- La clausola `throw` del metodo ridefinito può essere differente da quella del metodo ridefinente a patto che l'eccezione del metodo ridefinente sia la stessa o un sottotipo di quella del metodo

Ereditarietà e ridefinizione dei membri: gli attributi

- Gli attributi della superclasse possono essere nascosti se all'interno della classe derivata viene definito un altro attributo con lo stesso nome
- Per accedere all'attributo nascosto della superclasse si può
 - Utilizzare il modificatore super
 - Farlo attraverso l'utilizzo di altri membri della superclasse

Ereditarietà e ridefinizione dei membri

- La parola chiave `super` è disponibile per tutti i membri non statici
- Un metodo può essere ridefinito solo se è accessibile



Ogni oggetto di classe Counter2 è anche implicitamente di classe Counter ma non viceversa un Counter è meno ricco di un Counter2

la classe derivata mantiene l'interfaccia di accesso della classe base anche se, naturalmente, può specializzarla, aggiungendo nuovi metodi

Quindi, l'ereditarietà è più di un semplice “riuso di codice”: riusa l'astrazione

**Counter2 può essere usato al posto di un Counter se necessario
Ogni Counter2 è anche un Counter**

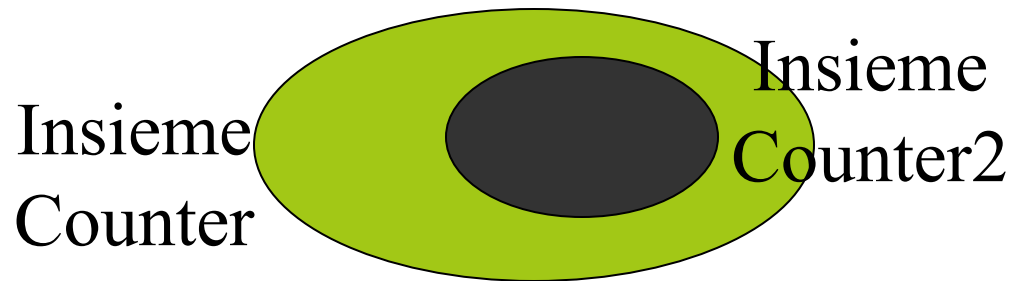
EREDITARIETÀ: CONSEGUENZE

- E' possibile usare un Counter2 al posto di un Counter senza che il sistema se ne accorga

```
public class Esempio6 {  
    public static void main(String args[]) {  
        Counter c1 = new Counter(10);  
        Counter2 c2 = new Counter2(20);  
        c2.dec();           // OK: c2 è un Counter2  
        // c1.dec();        // NO: c1 è solo un Counter  
        c1=c2;              // OK: c2 è anche un Counter  
        // c2=c1;          // NO: c1 è solo un Counter  
    }  
}
```

EREDITARIETÀ: CONSEGUENZE

- Dunque, la classe Counter2 definisce un sottotipo della classe Counter
- Gli oggetti di classe Counter sono compatibili con gli oggetti di classe Counter2 (perché la classe Counter2 è inclusa nella classe Counter)
- ma non viceversa



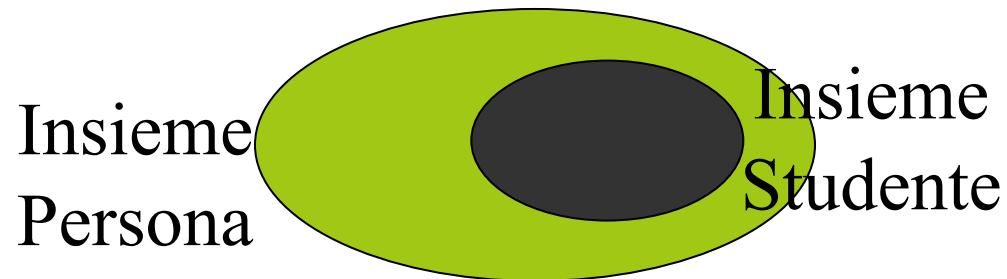
- Ovunque si possa usare un Counter, lì si può usare un Counter2 (ma non viceversa) sottinsieme dei Counter2

EREDITARIETÀ: CONSEGUENZE

- Dire che ogni Counter2 è anche un Counter significa dire che l'insieme dei Counter2 è un sottoinsieme dell'insieme dei Counter
- Se questo è vero nella realtà, la classificazione è aderente alla realtà del mondo
- Se invece è falso, questa classificazione nega la realtà del mondo
- Non è un buon modello del mondo
- Può produrre assurdità e inconsistenze

ESEMPIO

- Una classe Persona e una sottoclasse Studente

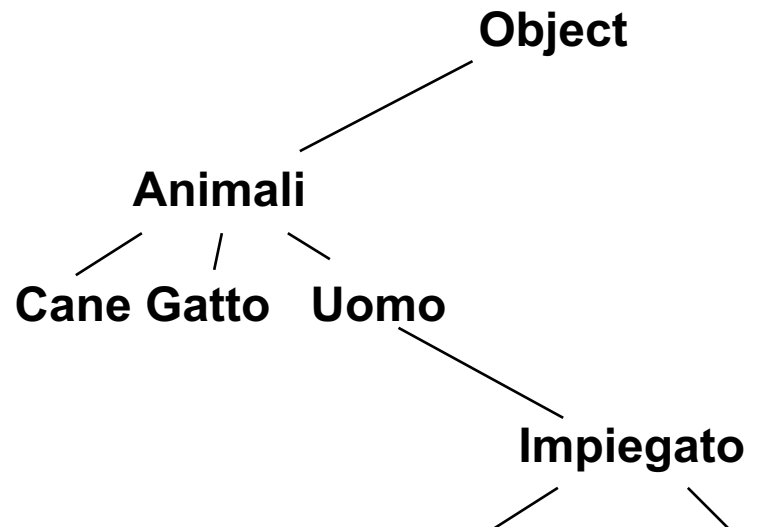
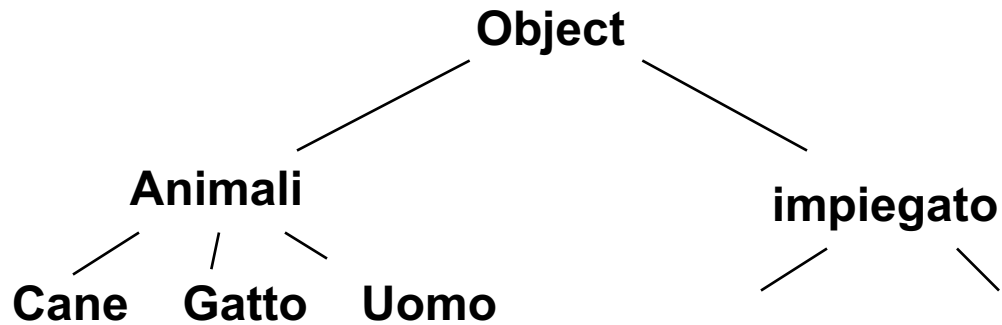


- è aderente alla realtà, perché è vero nel mondo reale che tutti gli studenti sono persone
- compatibilità di tipo: potremo usare uno studente (che è anche una persona) ovunque sia richiesta una generica persona
- ma non viceversa: se serve uno studente, non ci si può accontentare di una generica persona.

Alcune riflessioni su l'ereditarietà

- Com'è possibile che il metodo `toString()` possa essere automaticamente invocato?
- Come fa il compilatore Java a capire quale metodo è `toString`?
- Il segreto è nella classe *Object!*
 - Esiste in java una classe chiamata "Object"
(`java.lang.Object.`) quando noi scriviamo una qualsiasi classe in realtà deriviamo questa classe dalla classe "Object"

Object Hierarchy



Classe java.lang.Object

- Effetti positivi: sappiamo che tutte le classi hanno questi metodi
- Effetti negativi: questi metodi fanno solo raramente ciò che noi desideriamo
- Per risolvere questi problemi :
 - noi riscriviamo il metodo della classe Object
 - "Overriding" un metodo significa riscrivere il metodo nella nostra classe
 - la riscrittura del metodo farà ciò che noi vogliamo
 - noi dobbiamo usare esattamente la stessa signature del metodo della classe object
- Andrebbe sempre fatto l'overriding dei seguenti metodi:
 - equals()
 - toString()

java.lang.Object - equals()

Dall' API java.lang.Object:

```
public boolean equals(Object obj)
```

- Il metodo di default equals (quello della classe Object) usa "=="!!
- Per ogni coppia di riferimenti x e y x.equals(y) ritorna **true** se e solo se x, e y si riferiscono allo stesso oggetto, come x==y; **false** altrimenti.

java.lang.Object - toString()

```
public String toString()
```

- ritorna una stringa che e' la rappresentazione dell'oggetto. In generale, il metodo toString ritorna una stringa che "testualmente rappresenta" l'oggetto.
- Il risultato dovrebbe essere conciso ma informativo per una persona che legge.
- **E' raccomandato che tutte le classi facciano l'override di questo metodo.**
- Il metodo toString per la classe Object ritorna una stringa consistente nel nome della classe di cui l'oggetto e' una istanza, il carattere at '@', un esadecimale senza segno che e' la rappresentazione del codice hash dell'oggetto.
- In altre parole, questa stringa ritorna una stringa uguale al valore di:
`getClass().getName() + '@' + Integer.toHexString(hashCode())`

class java.lang.Object

Alcuni metodi di java.lang.Object:

- ▣ **clone()**

- ▣ crea un clone dell'oggetto.
- ▣ E' utilizzato per creare una copia "vera" dell'oggetto

- ▣ **equals(Object)**

- ▣ confronta due oggetti

- ▣ **getClass()**

- ▣ ritorna la classe di un oggetto.

- ▣ **toString()**

- ▣ ritorna una stringa che rappresenta il valore
- ▣ usato implicitamente da println() per produrre una rappresentazione testuale dell'oggetto